RELATÓRIO DE EVIDÊNCIAS E DOCUMENTAÇÃO DE TESTES

Módulo de Cadastro de Voluntários — VolunJá API

1. Visão Geral

Funcionalidade / requisito	Cadastro de voluntários, validação dos dados, persistência, proteção de senha e emissão de sessão JWT.
Módulo testado	app.voluntarios (service, repository, router, schemas e main).
Arquivo de testes	tests/auth/test_voluntarios.py
Framework e recursos	Pytest, FastAPI TestClient, unittest.mock, PyJWT e Argon2.
Resultado geral informado	94 testes aprovados em 11,08 segundos na suíte completa do projeto.
Cobertura total do projeto	87,92% (61 linhas não cobertas de 505; mínimo exigido: 80%).
Cobertura do módulo voluntários	100% em main.py, repository.py, router.py, schemas.py e service.py.
Ambiente reportado	Linux — Python 3.14.2.

2. Matriz de Rastreabilidade dos Casos de Teste

A matriz abaixo organiza os cenários do arquivo de testes por camada. Os testes parametrizados são discriminados por variação, pois cada entrada gera uma execução individual do cenário.

ID	Método / cenário	Resultado esperado	Status
CT-01	test_cadastro_retorna_dto_publico_com_perfil_fixo	Cadastro válido; DTO público, perfil Voluntario, Bearer e validade de 60 min.	**APROVADO***
CT-02	test_cadastro_emite_jwt_de_60_minutos_sem_dados_sensiveis	JWT com sub, e-mail, perfil e expiração de 60 min; sem senha/hash.	**APROVADO***
CT-03	test_cadastro_normaliza_email_para_minusculas	E-mail armazenado/localizado em letras minúsculas.	**APROVADO***
CT-04	test_cadastro_rejeita_e_nao_persiste [e-mail duplicado]	Lança EmailJaCadastradoException e não persiste novo registro.	**APROVADO***
CT-05	test_cadastro_rejeita_e_nao_persiste [identificador duplicado]	Lança IdentificacaoJaCadastradaException e não persiste novo registro.	**APROVADO***
CT-06	test_cadastro_rejeita_e_nao_persiste [sem consentimento]	Lança ConsentimentoObrigatorioException e não persiste novo registro.	**APROVADO***
CT-07	test_senha_e_armazenada_como_hash_argon2	Senha salva como hash Argon2; senha correta valida e incorreta falha.	**APROVADO***
CT-08	test_repositorio_buscar_por_email_encontrado	Retorna VoluntarioEntity e normaliza e-mail no parâmetro SQL.	**APROVADO***
CT-09	test_repositorio_converte_flags_para_bool	Converte consentimento_termos e ativo para valores booleanos.	**APROVADO***
CT-10	test_repositorio_buscar_por_email_nao_encontrado	Retorna None quando o e-mail não existe.	**APROVADO***
CT-11	test_repositorio_buscar_por_identificador_encontrado	Localiza identificador e remove espaços externos.	**APROVADO***
CT-12	test_repositorio_buscar_por_identificador_nao_encontrado	Retorna None quando o identificador não existe.	**APROVADO***
CT-13	test_repositorio_salvar_executa_insert_e_commit	Executa INSERT, normaliza e-mail e confirma transação com commit.	**APROVADO***
CT-14	test_repositorio_sem_conexao_nao_acessa_banco	Sem conexão: buscas retornam None e salvar retorna a entidade recebida.	**APROVADO***
CT-15	test_router_cadastro_retorna_201_sem_expor_senha	POST retorna HTTP 201; resposta não expõe senha.	**APROVADO***

ID	Método / cenário	Resultado esperado	Status
CT-16	test_router_converte_excecoes_de_negocio [e-mail]	E-mail duplicado convertido em HTTP 409.	**APROVADO***
CT-17	test_router_converte_excecoes_de_negocio [identificador]	Identificador duplicado convertido em HTTP 409.	**APROVADO***
CT-18	test_router_converte_excecoes_de_negocio [consentimento]	Ausência de consentimento convertida em HTTP 400.	**APROVADO***
CT-19	test_router_rejeita_entrada_invalida_com_422 [senha curta]	Entrada inválida rejeitada com HTTP 422.	**APROVADO***
CT-20	test_router_rejeita_entrada_invalida_com_422 [e-mail inválido]	Entrada inválida rejeitada com HTTP 422.	**APROVADO***
CT-21	test_router_rejeita_entrada_invalida_com_422 [nome curto]	Entrada inválida rejeitada com HTTP 422.	**APROVADO***
CT-22	test_router_rejeita_entrada_invalida_com_422 [identificador curto]	Entrada inválida rejeitada com HTTP 422.	**APROVADO***
CT-23	test_router_rejeita_corpo_sem_consentimento_com_422	Corpo sem campo obrigatório é rejeitado com HTTP 422.	**APROVADO***
CT-24	test_router_get_db_abre_e_fecha_conexao	Dependência disponibiliza conexão e fecha ao encerrar o gerador.	**APROVADO***
CT-25	test_router_get_voluntario_service_monta_service	Dependência instancia VoluntarioService.	**APROVADO***
CT-26	test_main_registra_rota_de_voluntarios	Aplicação FastAPI registra POST /api/v1/voluntarios no OpenAPI.	**APROVADO***

*Status individual indicado como aprovado com base no resultado informado de 94 testes aprovados na suíte completa e na cobertura de 100% dos arquivos do módulo app/voluntarios. O console fornecido não apresenta um resumo isolado da execução deste arquivo; portanto, não é possível atribuir a ele, de forma independente, 26 aprovações observadas.

3. Detalhamento dos Cenários de Teste

3.1. Serviço de cadastro

Cenário 1 — Cadastro válido e retorno do DTO público

• Dado que são fornecidos nome, e-mail, identificador, senha e consentimento válidos;

• Quando o serviço processa o cadastro;

• Então deve retornar os dados públicos do voluntário, com perfil fixo “Voluntario”;

• E deve informar token Bearer e expiração de 60 minutos;

• E o registro deve ser persistido como ativo e com consentimento.

Cenário 2 — Emissão e conteúdo do JWT

• Dado um cadastro realizado com sucesso;

• Quando o token JWT é decodificado;

• Então o campo sub deve corresponder ao ID do voluntário;

• E o token deve conter e-mail e perfil, com validade de uma hora;

• E não deve conter password nem password_hash.

Cenário 3 — Normalização do e-mail

• Dado um e-mail informado com letras maiúsculas;

• Quando o cadastro é realizado;

• Então o e-mail deve ser normalizado para letras minúsculas.

Cenário 4 — Rejeição de duplicidade e falta de consentimento

• Dado um e-mail já cadastrado, um identificador já utilizado ou consentimento falso;

• Quando o cadastro é solicitado;

• Então o serviço deve lançar a exceção de negócio correspondente;

• E nenhum novo registro deve ser persistido.

Cenário 5 — Armazenamento seguro da senha

• Dado uma senha válida fornecida no cadastro;

• Quando o serviço persiste o voluntário;

• Então a senha não deve ser armazenada em texto puro;

• E o hash Argon2 deve validar a senha original;

• E uma senha incorreta deve falhar na verificação.

3.2. Repositório

Cenário 6 — Busca por e-mail

• Dado um registro retornado pelo banco;

• Quando o repositório busca pelo e-mail;

• Então deve converter o resultado em VoluntarioEntity e normalizar o e-mail;

• E deve retornar None quando não houver correspondência.

Cenário 7 — Conversão dos indicadores booleanos

• Dado valores numéricos 0 ou 1 vindos do banco;

• Quando a entidade é construída;

• Então consentimento_termos e ativo devem ser convertidos para bool.

Cenário 8 — Busca por identificador

• Dado um identificador com ou sem espaços externos;

• Quando o repositório executa a consulta;

• Então deve remover os espaços antes de consultar;

• E deve retornar None se o registro não for encontrado.

Cenário 9 — Persistência e ausência de conexão

• Dado uma entidade válida e uma conexão simulada;

• Quando salvar é chamado;

• Então o repositório deve executar INSERT e commit e retornar a entidade;

• E sem conexão, não deve acessar o banco e deve preservar o retorno definido pelo teste.

3.3. Router e aplicação

Cenário 10 — Cadastro via endpoint

• Dado um corpo de requisição válido;

• Quando é enviado POST para /api/v1/voluntarios;

• Então a API deve responder HTTP 201;

• E deve retornar perfil e token sem expor a senha.

Cenário 11 — Conversão de exceções de negócio

• Dado e-mail duplicado, identificador duplicado ou falta de consentimento;

• Quando o serviço lança a exceção correspondente;

• Então o router deve converter os erros para HTTP 409, HTTP 409 e HTTP 400, respectivamente;

• E o detalhe da resposta deve refletir a mensagem da exceção.

Cenário 12 — Validação de entrada

• Dado senha curta, e-mail inválido, nome curto ou identificador curto;

• Quando a requisição é enviada;

• Então a API deve rejeitar os dados com HTTP 422;

• E um corpo sem consentimento também deve ser rejeitado com HTTP 422.

Cenário 13 — Ciclo de vida da conexão e montagem do serviço

• Dado uma conexão simulada;

• Quando get_db disponibiliza a conexão e o gerador é encerrado;

• Então a conexão deve ser fechada;

• E get_voluntario_service deve montar uma instância de VoluntarioService.

Cenário 14 — Registro da rota na aplicação

• Dado a aplicação principal do módulo;

• Quando o documento OpenAPI é consultado;

• Então deve existir /api/v1/voluntarios com o método POST.

4. Evidências de Execução e Cobertura

O resultado de console fornecido refere-se à suíte completa do projeto, não exclusivamente ao arquivo de testes de voluntários. Ele registra 94 testes aprovados em 11,08 segundos, com cobertura total de 87,92%, acima do requisito mínimo de 80%.

```
platform linux -- Python 3.14.2-final-0
94 passed in 11.08s
```

```
TOTAL: 505 statements | 61 missed | 88% (87.92%)
Required test coverage of 80% reached.
Coverage XML written to file coverage.xml
```

4.1. Cobertura dos arquivos do módulo app/voluntarios

Arquivo	Stmts	Miss	Cover	Resultado
app/voluntarios/main.py	6	0	**100%**	**Completa**
app/voluntarios/repository.py	38	0	**100%**	**Completa**
app/voluntarios/router.py	25	0	**100%**	**Completa**
app/voluntarios/schemas.py	13	0	**100%**	**Completa**
app/voluntarios/service.py	38	0	**100%**	**Completa**

5. Comandos de reprodução

Para executar especificamente os testes do módulo de voluntários:

```
pytest tests/auth/test_voluntarios.py -v
```

Para executar a suíte completa com relatório de cobertura:

```
pytest --cov=app --cov-report=term-missing
```

O relatório de cobertura apresentado pelo usuário informa também a geração de coverage.xml. A execução específica do arquivo pode ser usada para obter evidência isolada do módulo, caso seja necessário anexar um resultado individual.

6. Conclusão

Com base no código de testes e no relatório de cobertura fornecidos, o módulo app/voluntarios apresenta cobertura de 100% em todos os cinco arquivos listados. A suíte completa do projeto registrou 94 testes aprovados, em 11,08 segundos, e cobertura global de 87,92%, superando o limite mínimo configurado de 80%.

A matriz documenta 26 cenários/variações derivados das funções de teste do arquivo tests/auth/test_voluntarios.py. Como o log compartilhado mostra apenas o total global da suíte, recomenda-se executar esse arquivo isoladamente se for necessária uma evidência de aprovação exclusiva dos testes de voluntários.

Fontes utilizadas: código de testes tests/auth/test_voluntarios.py; relatório de cobertura e resultado geral de execução fornecidos pelo usuário; documento de referência “Relatório de Evidência e Documentação de Testes: Issue #05”.